Table 3.4 – Diagnostic d'implémentation de la génération contrôlée

Bloc	Preuve d'implémentation	Statut
Analyse requête	Noeud LangGraph `analyse_request`	Implémenté
Routage outils	Registre d'outils pgvector, Neo4j, recommandation hybride et synthèse globale	Implémenté
Construction contexte	Agrégation des résultats avant réponse finale	Implémenté
Génération	Client LLM OpenRouter avec fallback local	Implémenté
Critic	Vérification de l'appui contextuel et possibilité d'élargir la recherche	Implémenté
Score génération	À mesurer avec questions annotées et RAGAS dans le chapitre d'évaluation	Non mesuré ici

Source : outputs/memoire_stats/implementation_deep_current/13_agentic_generation_diagnostic.csv

La frontière est volontairement stricte. Côté retrieval, des métriques sont déjà disponibles. Côté génération, ce chapitre ne revendique pas encore une performance de factualité ou de complétude. Ces dimensions exigent un jeu de questions, des réponses de référence ou des critères annotés, puis une analyse avec RAGAS et une calibration du *critic*. Le bon discours d'implémentation est donc : la chaîne de génération est raccordée aux sources, mais la qualité de ses réponses reste à établir dans le chapitre suivant.

3.6 Synthèse du chapitre

L'implémentation aboutit à un système alimenté de bout en bout. Les données nettoyées couvrent 13 957 offres et 41 298 candidats. Le modèle SentenceTransformer fine-tuné domine les modèles de référence sur le classement sémantique. La base pgvector contient 72 643 embeddings synchronisés avec Neo4j et des index adaptés à la recherche hybride. Le graphe reconstruit contient 90 230 noeuds et 550 542 relations, principalement structurés autour des candidats, offres, compétences et référentiels. Enfin, l'orchestration Agentic RAG relie retrieval, graphe, contexte et génération contrôlée.

Les limites sont tout aussi importantes que les réussites. Les descriptions Louma-Jobs sont très peu renseignées, les compétences mélangent encore plusieurs natures d'information, les hubs du graphe doivent être normalisés pour ne pas dominer le score, et la génération n'a pas encore été évaluée comme réponse finale. Le chapitre d'évaluation devra donc se concentrer sur les seuils, le compromis rappel–précision, la calibration du *critic*, et la mesure de factualité des réponses générées.

Évaluation de la performance du système

4.1 Objectif et logique générale de l'évaluation

Ce chapitre évalue empiriquement le système implémenté au chapitre précédent. L'objectif n'est pas seulement de produire des scores globaux, mais de répondre à une question centrale du mémoire : l'ajout d'un graphe de connaissances Neo4j apporte-t-il une amélioration mesurable par rapport à une recherche vectorielle fondée uniquement sur pgvector ? Cette question est décisive, car le graphe ajoute une complexité technique réelle : modélisation des nœuds et relations, chargement Neo4j, requêtes Cypher, enrichissement des résultats, maintenance d'une base supplémentaire. Son intérêt doit donc être justifié par un gain observable en qualité de classement, en explicabilité ou en contrôle métier.

L'évaluation a été conduite en trois niveaux. Le premier niveau porte sur le modèle d'embeddings, c'est-à-dire la capacité du SentenceTransformer fine-tuné à retrouver les paires offre–profil pertinentes dans un corpus de test. Le deuxième niveau porte sur l'ablation du moteur de recommandation : une configuration *pgvector seul* est comparée à une configuration *pgvector + graphe*. Le troisième niveau porte sur les limites opérationnelles observées, notamment la stabilité de la connexion Neo4j et la portée réelle des métriques utilisées.

Les résultats présentés dans ce chapitre proviennent de deux artefacts exécutables : le benchmark d'embeddings sauvegardé dans `outputs/evaluation/embedding_benchmark.json` et le notebook d'ablation `notebooks/10_Evaluation_pgvector_vs_graph.ipynb`. Les sorties de ce notebook sont exportées dans `outputs/evaluation/pgvector_vs_graph` et les figures dans `rapport/figures/generated/evaluation`. Cette traçabilité est importante : les nombres présentés ici ne sont pas des valeurs théoriques, mais des sorties produites par le pipeline local.

4.2 Données et protocole expérimental

Le système dispose de 7 861 offres normalisées et de 1 105 profils candidats. Le benchmark d'embeddings utilise le jeu de test issu de la construction des paires contrastives offre–profil, soit 473 requêtes de test. Chaque requête correspond à une description structurée d'offre ou de profil, et le corpus contient les textes enrichis utilisés pour l'apprentissage contrastif. Les métriques calculées sont celles de la recherche d'information : Precision@K, Recall@K, NDCG@K et MRR@K.

Pour l'ablation *pgvector* contre *pgvector + graphe*, le protocole est différent. Un échantillon de 60 candidats est tiré aléatoirement avec la graine 42. Pour chaque candidat, pgvector retourne un pool initial de 30 offres candidates. Deux classements sont alors comparés :

- **pgvector seul** : l'ordre des offres est conservé tel qu'il est retourné par la recherche sémantique et lexicale dans PostgreSQL/pgvector ;
- **pgvector + graphe** : le même pool initial est enrichi par Neo4j, puis rerangé par le score hybride combinant similarité sémantique, compatibilité de compétences, compatibilité de niveau NCF et alignement sectoriel.

Cette comparaison est volontairement réalisée sans génération LLM. Le modèle génératif OpenRouter intervient dans le chatbot pour formuler la réponse finale, mais il n'est pas mobilisé dans l'ablation. Cette séparation évite de confondre deux phénomènes : la qualité du classement des offres et la qualité rédactionnelle de la réponse.

La pertinence utilisée pour calculer les métriques d'ablation est un proxy construit à partir des métadonnées normalisées : chevauchement lexical entre le profil et l'offre, proximité sectorielle et compatibilité de niveau NCF. Cette approximation est acceptable pour comparer deux variantes internes sur le même échantillon, mais elle ne remplace pas une annotation humaine. Les résultats doivent donc être interprétés comme une mesure comparative de l'effet du graphe, non comme une mesure définitive de satisfaction utilisateur.

4.3 Évaluation du modèle d'embeddings

Le premier résultat concerne l'intérêt du fine-tuning du SentenceTransformer. Le tableau 4.1 présente les performances du modèle fine-tuné par rapport à plusieurs modèles généralistes ou multilingues. Le modèle spécialisé domine nettement les alternatives sur les métriques de retrieval.

Table 4.1 – Benchmark des modèles d'embeddings sur 473 requêtes de test

Modèle	NDCG@10	MRR@10	Recall@10	Latence ms/phrase
ST fine-tuné emploi-compétences	0.6336	0.5653	0.8520	32.77
MiniLM-L6 généraliste	0.1980	0.1603	0.3192	26.03
DistilUSE multilingue	0.1773	0.1434	0.2896	57.96
mE5-base multilingue	0.1505	0.1274	0.2262	295.76
ML-MiniLM-L12	0.1115	0.0922	0.1734	42.82
CamemBERT sentence	0.0904	0.0771	0.1332	144.15

Le modèle fine-tuné atteint un Recall@10 de 0.8520, contre 0.3192 pour MiniLM-L6 généraliste. L'écart est substantiel : le modèle spécialisé retrouve beaucoup plus souvent le document pertinent dans les dix premiers résultats. Le NDCG@10 passe également de 0.1980 à 0.6336, ce qui indique non seulement une meilleure récupération, mais aussi un meilleur ordre des résultats pertinents. La latence moyenne du modèle spécialisé reste contenue à 32.77 ms par phrase, ce qui est compatible avec une indexation ou une recherche interactive locale.

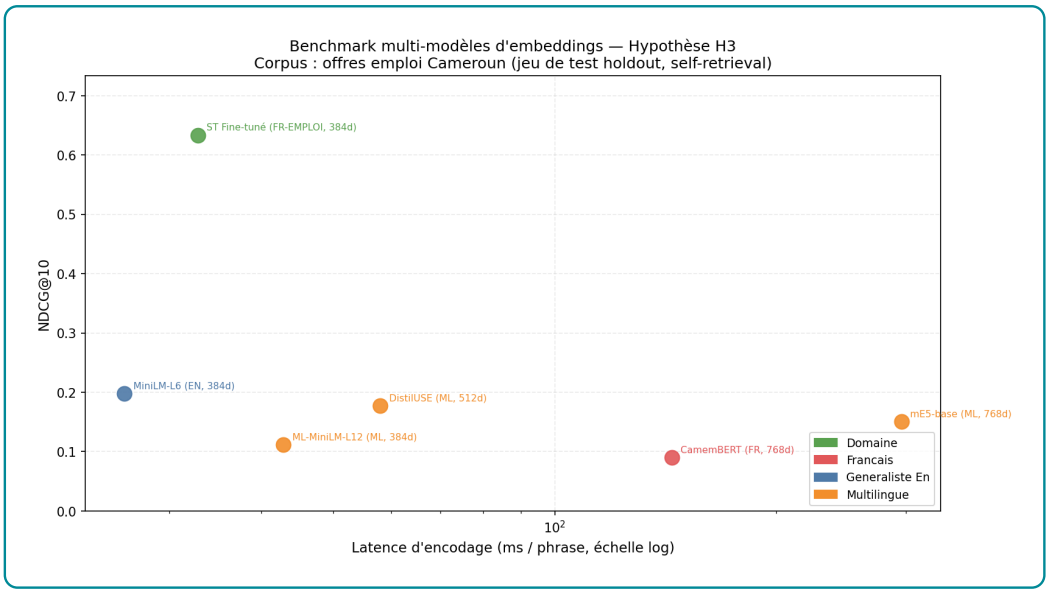

Figure 4.1 – Comparaison des modèles d'embeddings sur le jeu de test

Ce résultat valide le choix d'un encodeur adapté au domaine. Il montre aussi pourquoi une base vectorielle généraliste ne suffit pas : le vocabulaire emploi-compétences contient des formulations longues, des intitulés hybrides, des niveaux de formation et des compétences implicites. Le fine-tuning contrastif rapproche ces formulations dans l'espace vectoriel et améliore directement le retrieval.

4.4 Ablation pgvector seul contre pgvector + graphe

Le coeur de l'évaluation concerne l'apport de Neo4j. Le notebook d'ablation a demandé l'évaluation de 60 candidats. Sur cet échantillon, 55 requêtes ont été évaluées avec succès ; 5 requêtes ont échoué en raison d'une erreur interne Neo4j de type `CursorException` sur certaines relations. Cette information doit être conservée dans l'analyse : elle montre que la qualité d'un système GraphRAG ne dépend pas seulement du raisonnement algorithmique, mais aussi de l'intégrité physique et logique de la base graphe.

Table 4.2 – Comparaison retrieval : pgvector seul versus pgvector + graphe

Configuration	Precision@5	Recall@10	NDCG@10	MRR@10	HitRate@10
pgvector seul	0.2509	0.4599	0.3889	0.5602	0.9273
pgvector + graphe	0.3091	0.5557	0.4681	0.4647	0.7818
Différence	+0.0582	+0.0958	+0.0792	-0.0955	-0.1455

Les résultats sont instructifs parce qu'ils ne sont pas uniformément favorables au graphe. L'ajout de Neo4j améliore Precision@5, Recall@10 et NDCG@10. Autrement dit, lorsque l'on considère les cinq ou dix premiers résultats, l'enrichissement graphe permet de remonter davantage d'offres jugées pertinentes par le proxy métier. Le Recall@10 progresse de 0.4599 à 0.5557, soit un gain de 0.0958. Le NDCG@10 augmente également de 0.3889 à 0.4681, ce qui indique une meilleure organisation globale du top 10.

En revanche, le MRR@10 diminue de 0.5602 à 0.4647 et la Precision@1 passe de 0.4182 à 0.3636. Cela signifie que le graphe ne place pas toujours la première offre pertinente aussi haut que le classement

vectoriel initial. Ce résultat est important : le graphe améliore la couverture et la cohérence globale du classement, mais il peut dégrader le tout premier rang lorsque ses signaux structurels sont imparfaits, incomplets ou trop fortement pondérés.

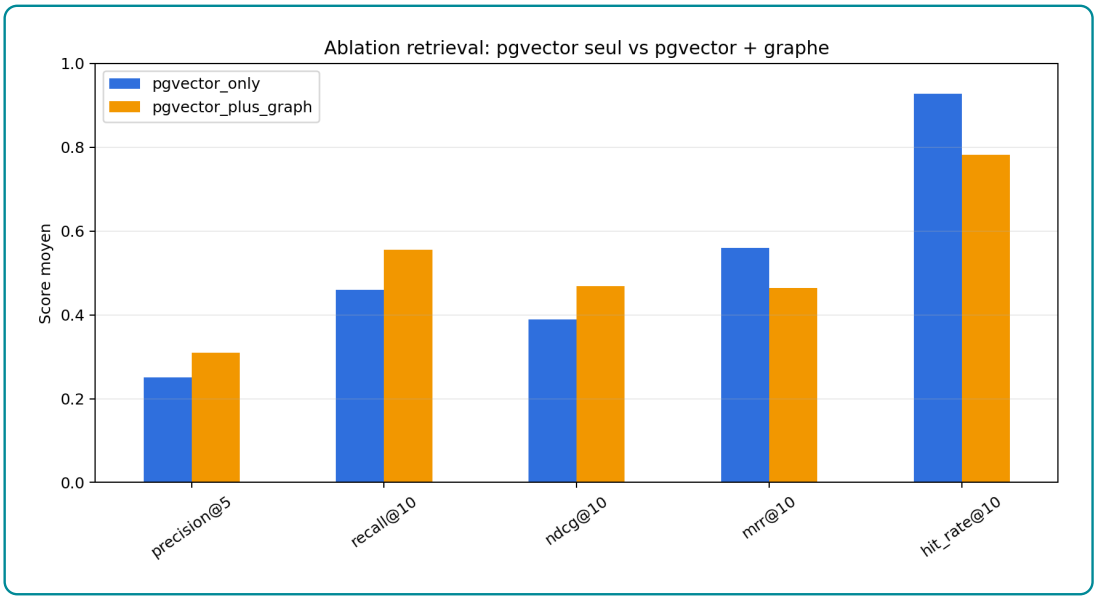

Figure 4.2 – Effet de l'enrichissement graphe sur les métriques de retrieval

La figure 4.2 confirme cette lecture. La configuration *pgvector + graphe* est supérieure sur les métriques orientées couverture, mais inférieure sur les métriques sensibles au premier rang. Dans un système de recommandation destiné au recrutement, cette distinction est opérationnellement importante. Si l'interface n'affiche qu'une seule offre, pgvector seul peut être plus robuste. Si l'interface affiche une liste courte accompagnée d'explications, le graphe devient plus utile car il enrichit le top 5 ou le top 10 avec des critères métier explicables.

4.5 Analyse du reranking par le graphe

Le reranking graphe déplace les offres dans le classement initial. Une offre qui était par exemple au rang 8 dans pgvector peut être remontée au rang 3 si elle présente une meilleure compatibilité NCF, un meilleur alignement sectoriel ou un meilleur score de compétences. Inversement, une offre très proche sémantiquement peut être rétrogradée si le niveau requis ou les signaux métier sont moins compatibles.

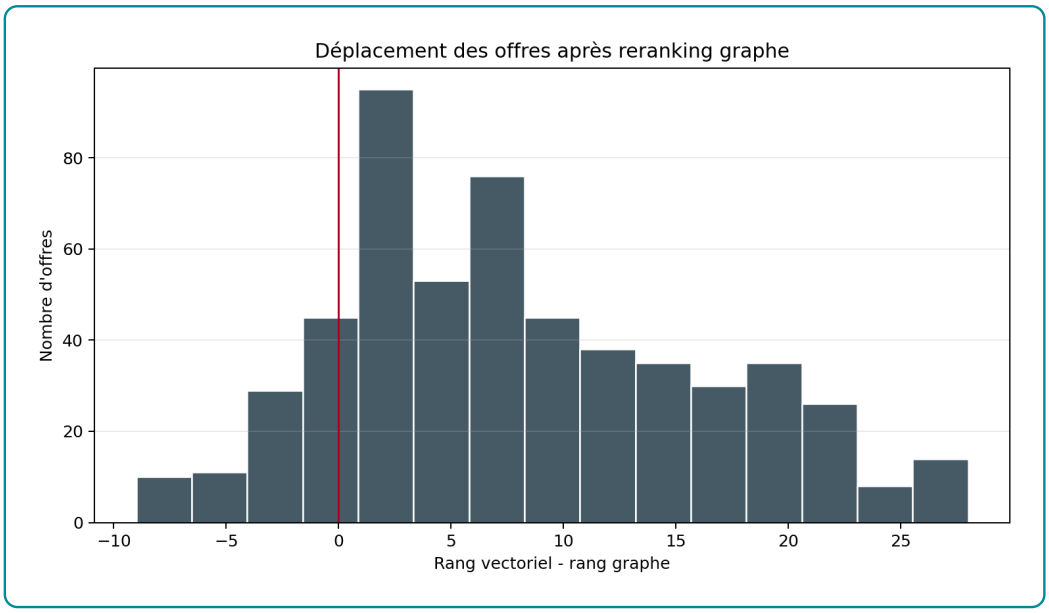

Figure 4.3 – Distribution des déplacements de rang après reranking par le graphe

Cette redistribution a deux effets. Le premier est positif : le système ne se limite plus à une proximité textuelle. Il peut corriger des cas où une offre ressemble lexicalement au profil mais ne correspond pas au niveau de formation ou au secteur recherché. Le second est plus problématique : si les relations du graphe sont incomplètes, le score structurel peut pénaliser de bonnes offres ou favoriser des offres seulement parce que leurs métadonnées sont mieux renseignées. Le graphe ajoute donc de l'information, mais il ajoute aussi une source d'erreur potentielle.

Ce résultat appelle une conclusion nuancée. Le graphe n'est pas une garantie automatique d'amélioration. Il améliore le système si les relations sont correctes, si les poids du score hybride sont bien calibrés et si l'objectif est de produire une liste explicable plutôt qu'un unique meilleur résultat. Dans le système actuel, l'apport est visible sur Recall@10 et NDCG@10, mais il reste insuffisamment stabilisé sur le rang 1.

4.6 Performance opérationnelle et stabilité

L'évaluation d'ablation a pris 38.7 secondes pour le coeur retrieval–reranking, hors coût complet de démarrage du notebook et de chargement du modèle. Le notebook exécuté par `nbconvert` a produit les résultats en environ 85 secondes en incluant le chargement du modèle SentenceTransformer, les connexions aux bases et l'export des figures. Cette durée reste acceptable pour une évaluation hors ligne, mais elle ne représente pas encore une latence utilisateur finale, car le chatbot complet ajoute la génération LLM via OpenRouter.

Le point critique est la stabilité Neo4j. Cinq candidats sur soixante ont échoué à cause d'une erreur interne `Neo.DatabaseError.Statement.ExecutionFailed` liée au page cache et à des relations corrompues. Le système a donc évalué 55 requêtes valides. Cette fragilité doit être traitée avant toute démonstration en production : il faut reconstruire ou vérifier la base Neo4j, contrôler les imports, et ajouter des tests d'intégrité sur les relations candidat–offre–compétence.

4.7 Interprétation par rapport aux hypothèses

Les résultats soutiennent partiellement l'hypothèse selon laquelle l'intégration d'un graphe améliore la recommandation. L'hypothèse est confirmée pour les métriques de couverture du top 10 : Recall@10 et NDCG@10 progressent lorsque Neo4j est utilisé pour enrichir et reranger les offres. En revanche, elle n'est pas confirmée pour le premier rang : Precision@1, MRR@10 et HitRate@10 diminuent. Le graphe apporte donc un gain structurel, mais ce gain n'est pas encore parfaitement aligné avec l'optimisation du meilleur résultat immédiat.

Sur le plan méthodologique, cette conclusion est plus utile qu'une affirmation générale. Elle montre où le graphe apporte de la valeur : explicabilité, couverture, prise en compte des niveaux et secteurs. Elle montre aussi où le système doit être amélioré : calibration des poids, complétude des relations, robustesse de Neo4j, et validation par annotation humaine.

4.8 Lecture des sorties graphiques de performance

Les figures du chapitre convergent vers une lecture prudente. La première sortie graphique, consacrée au benchmark d'embeddings, montre que le fine-tuning apporte le gain le plus net de toute l'évaluation : le modèle spécialisé domine les modèles généralistes sur NDCG@10, MRR@10 et Recall@10. Ce résultat indique que l'adaptation au vocabulaire emploi-compétences améliore directement la capacité du système à retrouver les correspondances pertinentes.

La deuxième sortie graphique, issue de l'ablation pgvector contre pgvector + graphe, confirme un effet plus nuancé. Le graphe améliore les métriques de couverture du top 5 et du top 10, mais il ne sécurise pas le premier rang. Cette différence est centrale pour l'interprétation métier : le graphe est utile lorsque l'utilisateur consulte une liste courte d'offres accompagnée d'explications, mais il reste insuffisamment calibré si l'objectif est de produire une unique meilleure offre.

La troisième sortie graphique, centrée sur les déplacements de rang, explique ce résultat. Neo4j ne se contente pas d'ajouter un score marginal; il réordonne réellement le pool initial retourné par pgvector. Cette intervention peut corriger des proximités purement lexicales, mais elle peut aussi rétrograder une bonne offre lorsque les relations de compétences, de niveau ou de secteur sont incomplètes. Le gain du graphe dépend donc de la qualité des relations et du réglage des poids du score hybride.

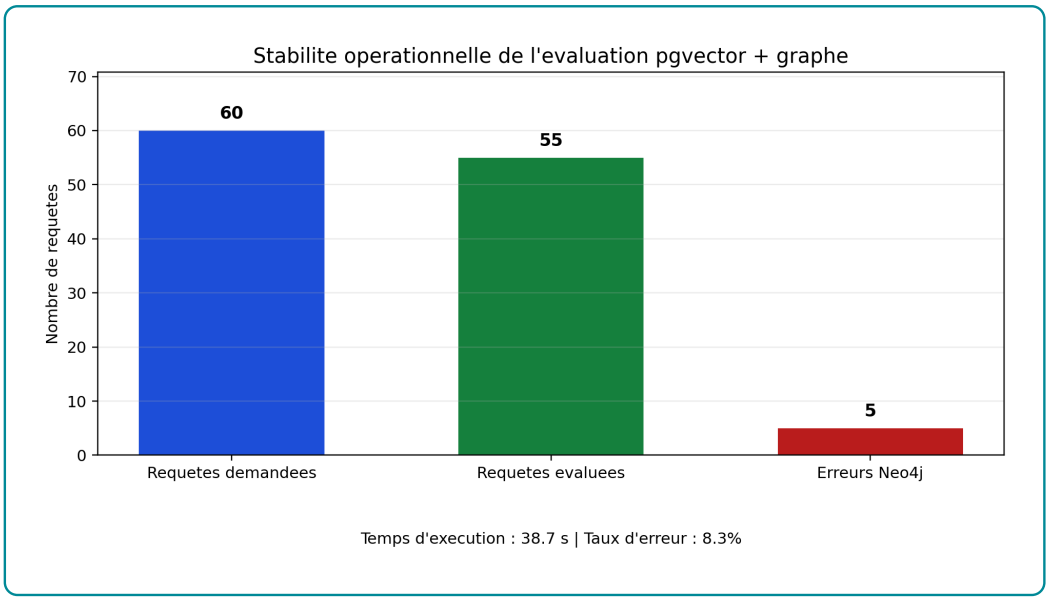

Figure 4.4 – Stabilité opérationnelle observée pendant l'évaluation d'ablation

La figure 4.4 complète cette lecture par la stabilité d'exécution. Sur 60 requêtes demandées, 55 ont été évaluées et 5 ont échoué à cause de Neo4j. Le taux d'erreur observé atteint donc 8.3%. Ce résultat n'annule pas les gains mesurés sur Recall@10 et NDCG@10, mais il empêche de conclure à une robustesse de production. Dans l'état actuel, la performance algorithmique du graphe existe, mais elle est conditionnée par un assainissement de la base Neo4j.

Enfin, aucune figure de calibration du critic n'est interprétée dans ce chapitre, car le dépôt ne contient pas encore de jeu réel `critic_calibration_cases.csv`. Le fichier disponible est seulement un gabarit, avec des lignes fictives. L'utiliser pour produire des courbes RAGAS donnerait des graphiques visibles, mais méthodologiquement faux. La calibration du critic doit donc rester une étape d'évaluation complémentaire, à exécuter uniquement après constitution d'un échantillon réel de questions, réponses, contextes récupérés et labels RAGAS ou humains.

4.9 Limites de l'évaluation

La principale limite est l'absence d'un jeu d'annotations humaines indiquant quelles offres sont réellement pertinentes pour chaque candidat. Le proxy utilisé combine les métadonnées disponibles, mais il ne capture pas toutes les dimensions du recrutement : motivation, expérience qualitative, disponibilité, préférences salariales, soft skills ou contraintes géographiques fines. Les métriques doivent donc être lues comme des indicateurs comparatifs internes.

La deuxième limite concerne le graphe lui-même. Les erreurs Neo4j observées indiquent que l'instance locale doit être auditée. Une base graphe partiellement corrompue ou incomplète peut biaiser l'évaluation du système hybride. Avant une évaluation finale à plus grande échelle, il faudra reconstruire la base, vérifier les nombres de nœuds et de relations, puis contrôler des chemins représentatifs : candidat–compétence, offre–compétence, offre–niveau, candidat–formation et métier–compétence.

La troisième limite concerne la génération LLM. Ce chapitre a volontairement isolé le retrieval et le reranking. La fidélité des réponses générées, la qualité des citations, la robustesse du critic et l'expérience

utilisateur nécessitent une évaluation complémentaire, idéalement avec un protocole d'annotation humaine ou un juge LLM contrôlé par des critères explicites.

4.10 Synthèse du chapitre

L'évaluation confirme d'abord l'intérêt du SentenceTransformer fine-tuné : il dépasse nettement les modèles généralistes sur le jeu de test emploi-compétences. Elle montre ensuite que Neo4j apporte un gain réel sur les métriques de couverture et d'ordre global du top 10, mais qu'il dégrade le tout premier rang dans l'état actuel du système. La conclusion n'est donc pas que le graphe est automatiquement supérieur à pgvector. La conclusion correcte est que le graphe enrichit le classement lorsqu'on cherche une liste explicable et structurée, mais qu'il exige une meilleure qualité de relations et une calibration plus fine pour optimiser le top 1.

Ces résultats orientent directement les recommandations du chapitre suivant : renforcer l'intégrité Neo4j, construire un jeu d'évaluation annoté, ajuster les poids du score hybride, et distinguer dans l'interface les cas où le système privilégie la proximité sémantique immédiate et ceux où il privilégie la compatibilité métier structurée.

Discussion et recommandations

Conclusion générale

Bibliographie

[1] Francesco Ricci et al., éd. *Recommender Systems Handbook*. Springer, 2011. DOI : 10.1007/978-0-387-85820-3.

[2] pgvector contributors. *pgvector : Open-source Vector Similarity Search for Postgres*. 2024. URL : https://github.com/pgvector/pgvector (visité le 04/05/2026).

[3] Aidan Hogan et al. "Knowledge Graphs". In : *ACM Computing Surveys* 54.4 (2021), p. 1-37. DOI : 10.1145/3447772.

[4] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474. URL : https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[5] Dale T. Mortensen et Christopher A. Pissarides. "Job Creation and Job Destruction in the Theory of Unemployment". In : *The Review of Economic Studies* 61.3 (1994), p. 397-415. DOI : 10.2307/2297896.

[6] Christopher A. Pissarides. *Equilibrium Unemployment Theory*. 2e éd. Cambridge, MA : MIT Press, 2000.

[7] George A. Akerlof. "The Market for "Lemons" : Quality Uncertainty and the Market Mechanism". In : *The Quarterly Journal of Economics* 84.3 (1970), p. 488-500. DOI : 10.2307/1879431.

[8] Michael Spence. "Job Market Signaling". In : *The Quarterly Journal of Economics* 87.3 (1973), p. 355-374. DOI : 10.2307/1882010.

[9] Gary S. Becker. *Human Capital : A Theoretical and Empirical Analysis, with Special Reference to Education*. New York : Columbia University Press, 1964.

[10] David H. Autor. "The "Task Approach" to Labor Markets : An Overview". In : *Journal for Labour Market Research* 46.3 (2013), p. 185-199. DOI : 10.1007/s12651-013-0128-z.

[11] Gediminas Adomavicius et Alexander Tuzhilin. "Toward the Next Generation of Recommender Systems : A Survey of the State-of-the-Art and Possible Extensions". In : *IEEE Transactions on Knowledge and Data Engineering* 17.6 (2005), p. 734-749. DOI : 10.1109/TKDE.2005.99.

[12] Michael J. Pazzani et Daniel Billsus. "Content-Based Recommendation Systems". In : *The Adaptive Web* (2007), p. 325-341. DOI : 10.1007/978-3-540-72079-9_10.

[13] Yehuda Koren, Robert Bell et Chris Volinsky. "Matrix Factorization Techniques for Recommender Systems". In : *Computer* 42.8 (2009), p. 30-37. DOI : 10.1109/MC.2009.263.

[14] Xiangnan He et al. "Neural Collaborative Filtering". In : *Proceedings of the 26th International Conference on World Wide Web*. 2017, p. 173-182.

[15] Robin Burke. "Hybrid Recommender Systems : Survey and Experiments". In : *User Modeling and User-Adapted Interaction* 12 (2002), p. 331-370. DOI : 10.1023/A:1021240730564.

[16] Chen Yang et al. "Modeling Two-way Selection Preference for Person-Job Fit". In : *Proceedings of the 16th ACM Conference on Recommender Systems*. 2022, p. 102-112.

[17] A. T. Wasi. "HRGraph : Leveraging LLMs for HR Data Knowledge Graphs with Information Propagation-based Job Recommendation". In : *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models*. 2024, p. 56-62.

[18] B. Yang et Z. Shen. "Knowledge Graph Construction and Talent Competency Prediction for Human Resource Management". In : *Alexandria Engineering Journal* 121 (2025), p. 223-235.

[19] Ashish Vaswani et al. "Attention Is All You Need". In : *Advances in Neural Information Processing Systems*. 2017, p. 5998-6008.

[20] Tomas Mikolov et al. "Distributed Representations of Words and Phrases and their Compositionality". In : *Advances in Neural Information Processing Systems*. T. 26. 2013. URL : https://proceedings.neurips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.

[21] Dzmitry Bahdanau, Kyunghyun Cho et Yoshua Bengio. "Neural Machine Translation by Jointly Learning to Align and Translate". In : *International Conference on Learning Representations*. 2015. URL : https://arxiv.org/abs/1409.0473.

[22] Jacob Devlin et al. "BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding". In : *Proceedings of NAACL-HLT*. 2019, p. 4171-4186. URL : https://aclanthology.org/N19-1423/.

[23] Nils Reimers et Iryna Gurevych. "Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks". In : *Proceedings of EMNLP-IJCNLP*. 2019, p. 3982-3992. URL : https://aclanthology.org/D19-1410/.

[24] Tom B. Brown et al. "Language Models are Few-Shot Learners". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 1877-1901. URL : https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

[25] Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In : *Journal of Machine Learning Research* 21.140 (2020), p. 1-67. URL : https://www.jmlr.org/papers/v21/20-074.html.

[26] Long Ouyang et al. "Training Language Models to Follow Instructions with Human Feedback". In : *Advances in Neural Information Processing Systems* 35 (2022), p. 27730-27744. URL : https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.

[27] Nandan THAKUR et al. "BEIR : A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models". In : *Advances in Neural Information Processing Systems*. T. 34. 2021, p. 28974-28989. URL : https://arxiv.org/abs/2104.08663.

[28] Niklas MUENNIGHOFF et al. "MTEB : Massive Text Embedding Benchmark". In : *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, p. 2014-2037. URL : https://aclanthology.org/2023.eacl-main.148/.

[29] Yu A. MALKOV et D. A. YASHUNIN. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), p. 824-836. DOI : 10.1109/TPAMI.2018.2889473.

[30] Jeff JOHNSON, Matthijs DOUZE et Hervé JÉGOU. "Billion-scale Similarity Search with GPUs". In : *IEEE Transactions on Big Data* 7.3 (2019), p. 535-547. DOI : 10.1109/TBDATA.2019.2921572.

[31] Thomas N. KIPF et Max WELLING. "Semi-Supervised Classification with Graph Convolutional Networks". In : *International Conference on Learning Representations*. 2017.

[32] Petar VELIČKOVIĆ et al. "Graph Attention Networks". In : *International Conference on Learning Representations*. 2018.

[33] Xiangnan HE et al. "LightGCN : Simplifying and Powering Graph Convolution Network for Recommendation". In : *Proceedings of SIGIR*. 2020, p. 639-648.

[34] Yunfan GAO et al. *Retrieval-Augmented Generation for Large Language Models : A Survey*. 2023. arXiv : 2312.10997 [cs.CL]. URL : https://arxiv.org/abs/2312.10997.

[35] Shunyu YAO et al. "ReAct : Synergizing Reasoning and Acting in Language Models". In : *International Conference on Learning Representations*. 2023. URL : https://openreview.net/forum?id=WE_vluYUL-X.

[36] Akari ASAI et al. "Self-RAG : Learning to Retrieve, Generate, and Critique through Self-Reflection". In : *International Conference on Learning Representations*. 2024. URL : https://openreview.net/forum?id=hSyW5go0v8.

[37] Shahul ES et al. "RAGAS : Automated Evaluation of Retrieval Augmented Generation". In : *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics : System Demonstrations*. 2024, p. 150-158. URL : https://aclanthology.org/2024.eacl-demo.16/.

[38] Yang LIU et al. "G-Eval : NLG Evaluation using GPT-4 with Better Human Alignment". In : *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, p. 2511-2522. URL : https://aclanthology.org/2023.emnlp-main.153/.

[39] William L. HAMILTON. *Graph Representation Learning*. Morgan & Claypool Publishers, 2020. DOI : 10.2200/S01045ED1V01Y202009AIM046.

Annexes

Table A.1 – Labels de nœuds et relations structurantes du graphe

Bloc	Labels de nœuds	Relations principales et interprétation
Données d'emploi	`OffreEmploi`, `Candidat`, `Employeur`, `Secteur`, `Localisation`	`PUBLIEE_PAR` relie une offre à son employeur ; `DANS_SECTEUR` la rattache à un secteur ; `LOCALISEE_A` indique la ville ou zone d'exercice ; `A_NIVEAU` relie le candidat à son niveau de formation.
Compétences et métiers	`Compétence`, `Métier`, `GroupeCompétences`, `GroupeISCO`	`NECESSITE` relie un métier aux compétences attendues ; `PARTIE_DE` rattache une compétence à son groupe ; `CLASSIFIE_DANS` relie un métier à un groupe ISCO ; `PLUS_LARGE_QUE` représente les relations hiérarchiques entre compétences.
Référentiel MEPC	`GrandGroupeMEPC`, `SousGroupeMEPC`, `GroupeBaseMEPC`	`CONTIENT` représente la hiérarchie interne de la nomenclature ; `ALIGNE_AVEC` rapproche les groupes MEPC des groupes ISCO ; `CORRESPOND_MEPC` rattache un métier ESCO à un groupe de base MEPC.
Référentiel NCF	`NiveauFormationNCF`, `GrandDomaineNCF`, `DomaineSpécialiséNCF`, `DomaineDétailléNCF`	`CONTIENT` représente la hiérarchie formation ; `REQUIERT_NIVEAU_NCF` relie une offre au niveau demandé ; `A_FORMATION` rattache un candidat à son domaine de formation ; `PREPARE_POUR` relie un domaine de formation aux métiers visés.
Recommandation	`OffreEmploi`, `Candidat`, `Compétence`, `Métier`, `DomaineDétailléNCF`	Les chemins candidat–compétence–offre permettent d'identifier les compétences acquises et manquantes ; les chemins offre–niveau–formation servent à contrôler la compatibilité de qualification ; les chemins formation–métier soutiennent la proposition de montée en compétences.

Source : Auteur

Table A.2 – Volume d'entités indexées dans la base vectorielle pgvector

Type d'entité	Nombre	Source principale
`OFFRE_EMPLOI`	15 722	Scraping KmerAI
`COMPETENCE`	13 939	Référentiel ESCO v1.1
`METIER`	3 039	Référentiel ESCO v1.1
`CANDIDAT`	1 105	Base locale de profils demandeurs
`GROUPE_BASE_MEPC`	209	INS Cameroun - MEPC 2013
`DOMAINE_DETAILLE_NCF`	201	INS Cameroun - NCF 2017
Total	**34 215**	

Source : Auteur

Table des matières

Dédicace i

Remerciements ii

Liste des sigles et abréviations v

Liste des tableaux vi

Liste des figures vii

Avant-propos viii

Résumé ix

Abstract x

Introduction générale 1

I Cadre théorique et conceptuel 5

1 Fondements théoriques et revue de la littérature sur l'IA générative et les systèmes de recommandation 6

1.1 Concepts clés mobilisés dans le mémoire . . . 6

1.2 Fondements économiques et problématique d'appariement . . . 8

1.2.1 Marché du travail, recherche d'emploi et frictions . . . 8

1.2.2 Capital humain, tâches et inadéquation des compétences . . . 8

1.3 Systèmes de recommandation et person-job fit . . . 8

1.3.1 Approches par contenu, collaboratives et hybrides . . . 9

1.3.2 Spécificité du person-job fit . . . 9

1.4 IA générative et connaissances . . . 10

1.4.1 Grands modèles de langage et génération contrôlée . . . 10

1.4.2 Embeddings, recherche sémantique et bases vectorielles . . . 11

1.4.3 Graphes de connaissances et recommandation relationnelle . . . 11

1.5 RAG, GraphRAG et orchestration agentique . . . 11

1.5.1 De la RAG au GraphRAG . . . 11

1.5.2 Agentic RAG, critic et traçabilité . . . 12

1.6 Évaluation, acquis de la littérature et positionnement du mémoire . . . 13
1.6.1 Évaluer la performance et la factualité . . . 13
1.6.2 Acquis, manques et contribution du mémoire . . . 14

2 Source de données et Approche méthodologique 16
2.1 Cadre méthodologique et sources de données . . . 16
2.1.1 Positionnement méthodologique . . . 16
2.1.2 Sources de données et leur rôle dans le système . . . 19
2.1.3 Préparation et normalisation des données . . . 20
2.1.4 Alignement des référentiels métiers et formations . . . 21
2.2 Modélisation sémantique et structuration des connaissances . . . 21
2.2.1 Choix du modèle d'embeddings et fine-tuning . . . 21
2.2.2 Indexation vectorielle dans PostgreSQL et pgvector . . . 22
2.2.3 Construction du graphe de connaissances Neo4j . . . 22
2.2.4 Score hybride et logique de skill gap . . . 22
2.3 Orchestration agentique et génération contrôlée . . . 23
2.3.1 Architecture Agentic RAG et orchestration LangGraph . . . 23
2.3.2 Text2Cypher et rôle du LLM . . . 23
2.3.3 Interfaces et traçabilité . . . 24
2.4 Évaluation prévue, limites et synthèse méthodologique . . . 24
2.4.1 Protocole d'évaluation prévu . . . 24
2.4.2 Limites méthodologiques et précautions d'interprétation . . . 24
2.4.3 Synthèse de l'approche méthodologique . . . 25

II Présentation des résultats 26

3 Implémentation du système de recommandation 27
3.1 Description des données exploitées . . . 27
3.1.1 Sources des offres . . . 28
3.1.2 Localisation des opportunités . . . 28
3.1.3 Compétences et contenu descriptif . . . 29
3.2 Fine-tuning du modèle d'embeddings . . . 29
3.2.1 Justification du modèle de base . . . 29
3.2.2 Déroulement de l'entraînement . . . 30
3.2.3 Comparaison avec les modèles de référence . . . 32
3.3 Espace vectoriel et indexation pgvector . . . 32
3.3.1 Analyse de l'espace vectoriel . . . 33
3.3.2 Indexation dans pgvector . . . 33
3.3.3 Indicateurs de fonctionnement . . . 34
3.4 Graphe de connaissances Neo4j . . . 34
3.4.1 Schéma de la base de connaissances . . . 35

3.4.2 Analyse topologique du graphe 36
3.5 Implémentation Agentic RAG : retrieval et génération 37
3.5.1 Résultats côté retrieval 38
3.5.2 Résultats côté génération contrôlée 38
3.6 Synthèse du chapitre 39

4 Évaluation de la performance du système 40
4.1 Objectif et logique générale de l'évaluation 40
4.2 Données et protocole expérimental 40
4.3 Évaluation du modèle d'embeddings 41
4.4 Ablation pgvector seul contre pgvector + graphe 42
4.5 Analyse du reranking par le graphe 43
4.6 Performance opérationnelle et stabilité 44
4.7 Interprétation par rapport aux hypothèses 45
4.8 Lecture des sorties graphiques de performance 45
4.9 Limites de l'évaluation 46
4.10 Synthèse du chapitre 47

5 Discussion et recommandations 48

Conclusion générale I

Bibliographie I

Bibliographie II

A Annexes V